The Blueprint of Command-Line Automation

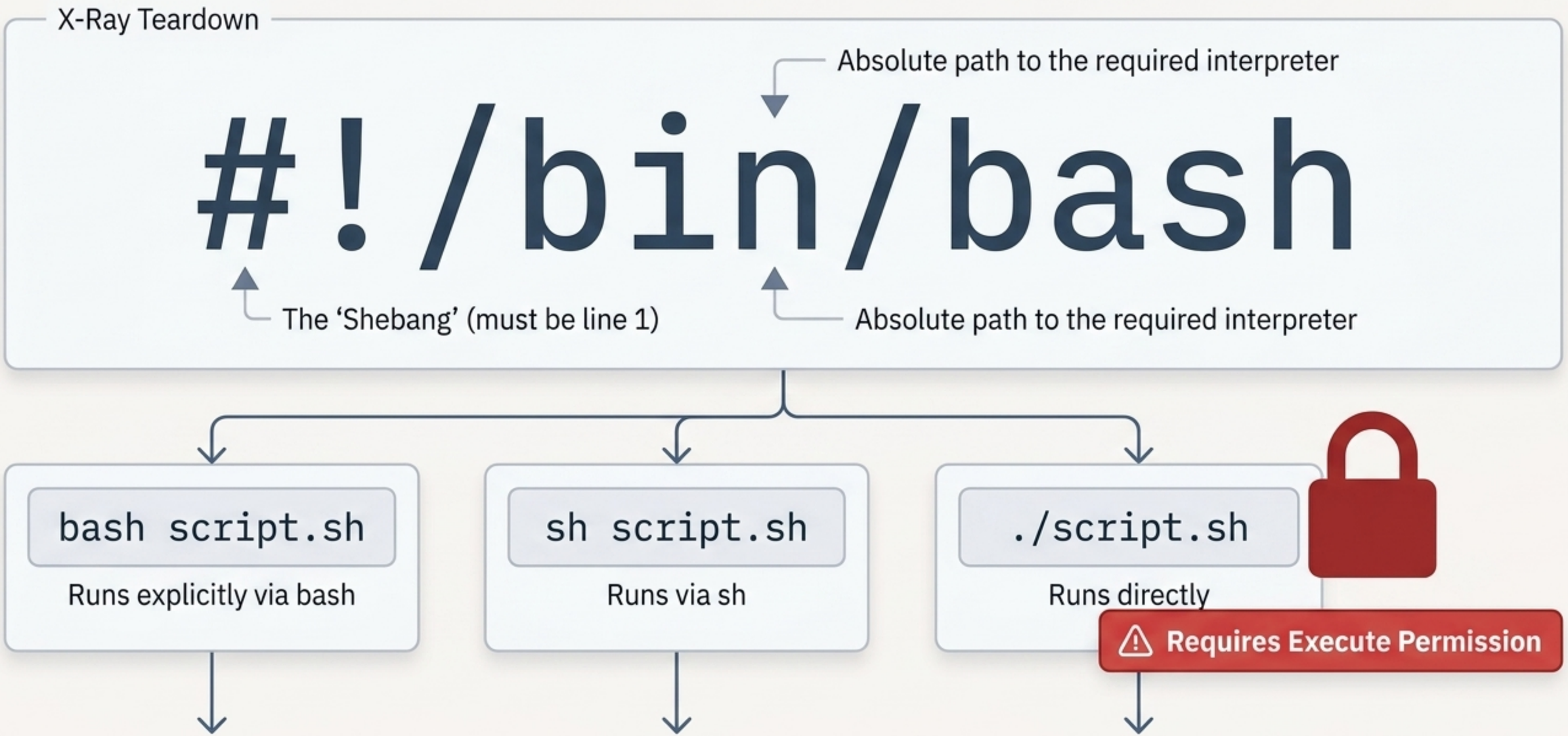
A Visual Guide to Bash Scripting: From fundamentals to modular architectures.

The Translation Bridge Between User and Kernel

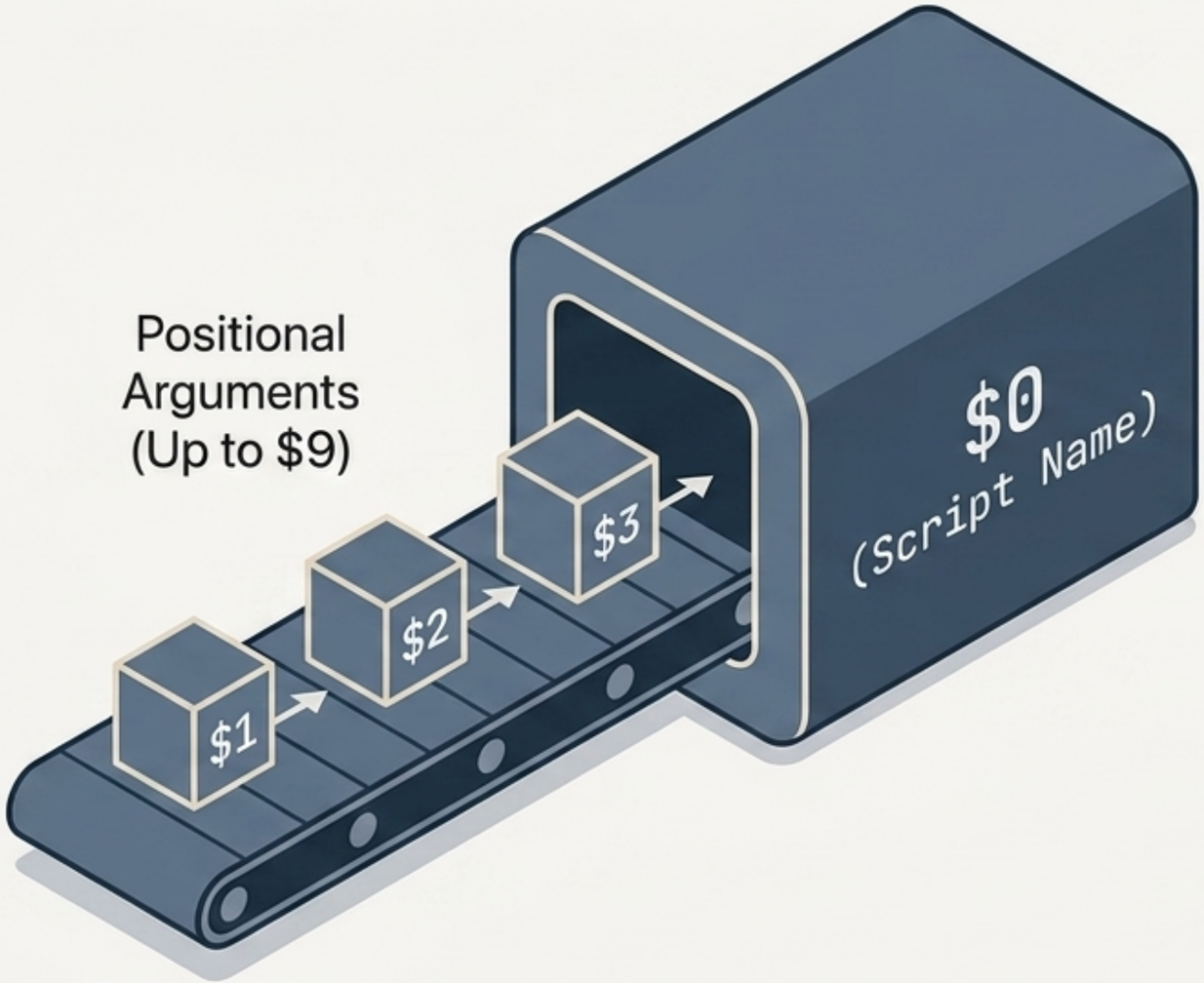
- **No compilation needed:** Evaluates logic, commands, and data directly.

The Engine: Automates processes, filters data, and combines standard Linux commands into repeatable workflows. Everything learned about Linux commands applies directly inside a Bash script.

Initializing Execution: The Shebang and Permissions



Feeding the Machine: Arguments and Special Variables



System Tracking Variables

Variable	Definition
$#	Total count of arguments passed.
$@	The full list of all arguments.
$$	The Process ID (PID) of the running script.
$?	Exit status of the last command (0 = success, 1 = failure).

Internal Storage: Variables, Strings, and Arrays

Variables

```
domain=$1
```

NO SPACES AROUND =

Spaces cause a ‘command not found’ error. Variables are treated as strings by default.

Arrays

```
domains=(value1 value2 value3)
```

Single or double quotes treat everything inside as one array element.

Forcing Arithmetic Mode: The Double Parentheses Rule

Calculations

+	-	*	/	%	++	--

Calculations: $(())

```
echo $((10 + 10)) → 20
```

Increment/Decrement: (())

```
((increase++))
```

Changes variable value

Measuring Strings

${#variable}

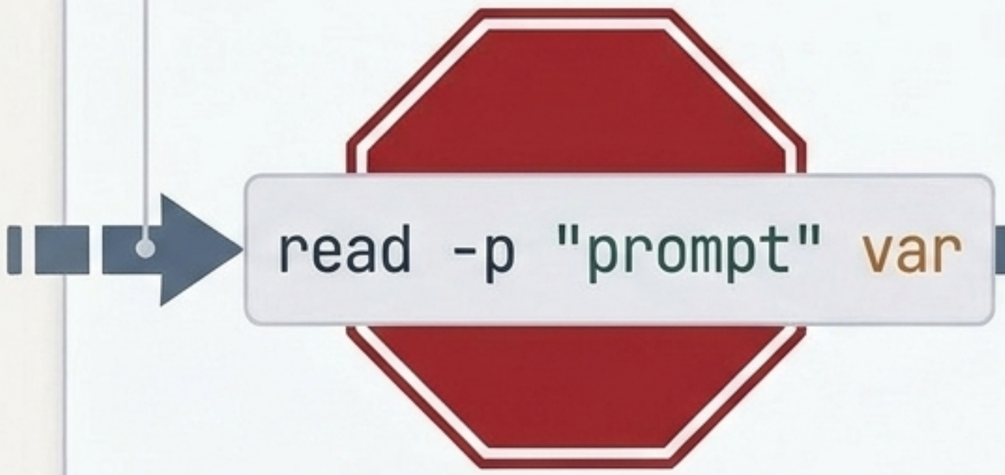
```
htb="HackTheBox"
echo ${#htb} ⟶ 10
```

Counts total characters,
does not evaluate math.

Interactivity: Controlling Input and Output

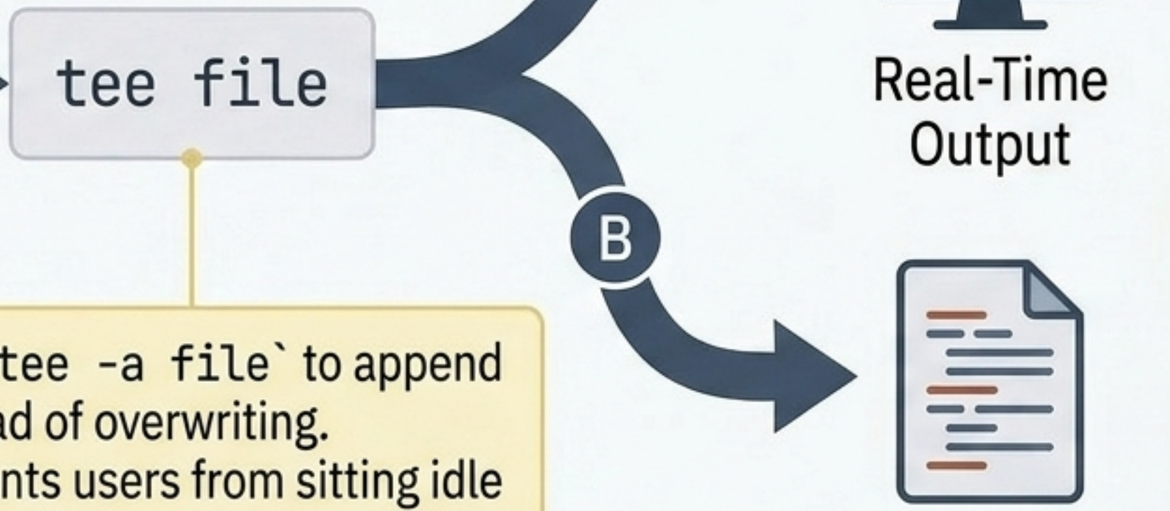
Two-Way Street

Inbound Traffic



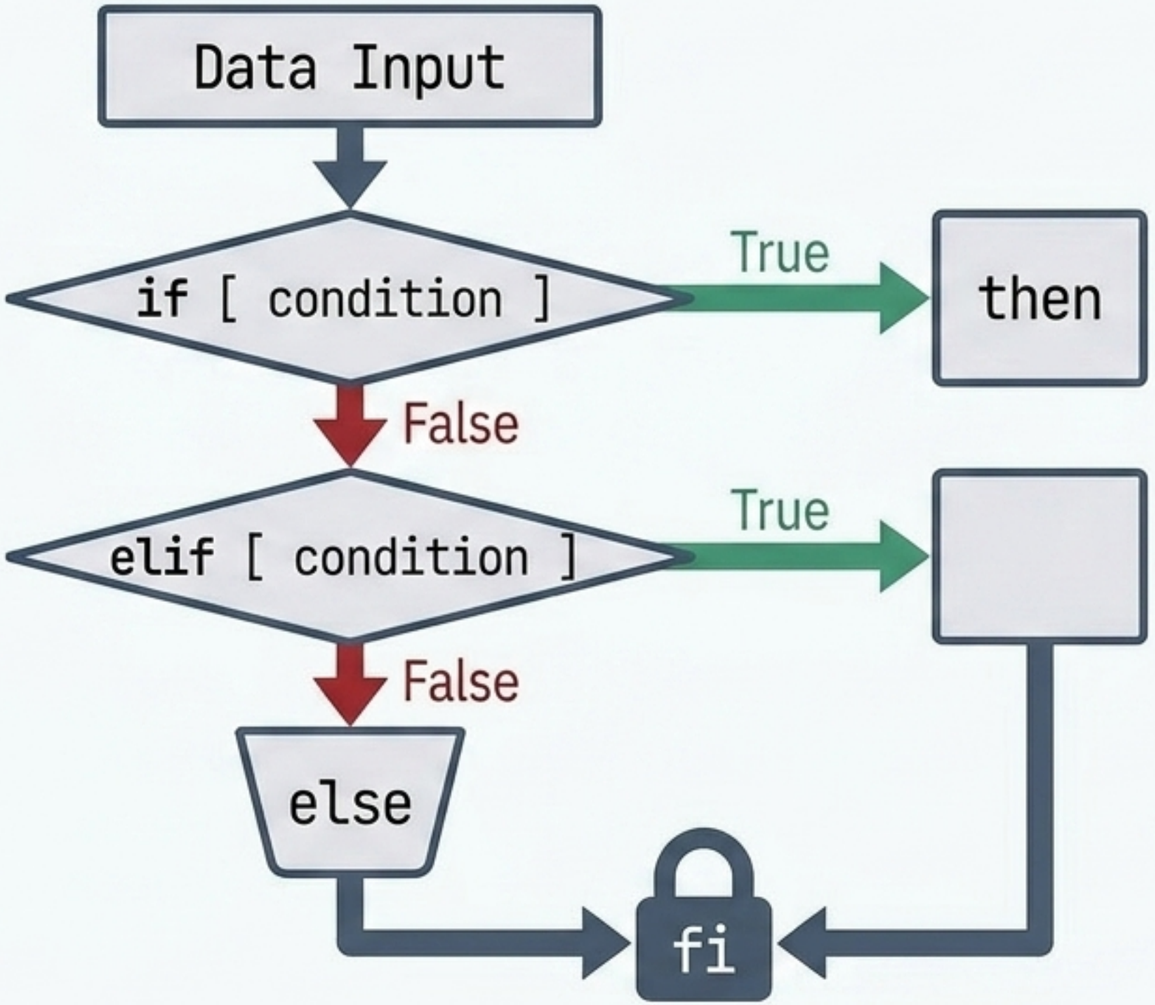
Pauses execution, displays prompt on the same line, and waits to store user input to control script flow.

Outbound Traffic



Use `tee -a file` to append instead of overwriting. Prevents users from sitting idle waiting for long-running scripts with no terminal feedback.

Sequential Decision Making: Conditional Flow



Conditions are evaluated in *strictly* sequence. Only the first match executes.

The Operator Matrix: Comparing Data Types

String		Integer		File	
==	equal	-eq	equal	-e	exists
!=	not equal	-ne	not equal	-f	regular file
		-lt	less than	-d	directory
< / >	ASCII alphabetical order	-le	less than/equal	-s	size > 0
-z	is empty/null	-gt	greater than	-r / -w / -x	read/write/execute permissions
-n	is not null	-ge	greater than/equal	-L	symlink
				-N	modified since last read
				-O / -G	user/group ownership

Note: ‘man ascii’ (7-bit encoding, values 0-127)

Advanced Conditions: Boolean Logic and Double Brackets

1. **String Math:** Enables the use of '<' and '>' operators safely.
2. **Logical Operators:** '&&' (AND - all must match), '||' (OR - any can match), '!' (NOT - negation).
3. **Boolean Output:** Returns `true`/`false` directly.

CRITICAL RULE: Always double-quote variables (e.g., "$1") inside conditions so Bash treats them safely as strings and avoids null errors.

Circular Execution: The Three Engines of Iteration

Iterates over a defined, finite list (numbers, strings, IPs, arrays).

```
for var in list; do ... done
```

Spins constantly as long as a condition is TRUE.

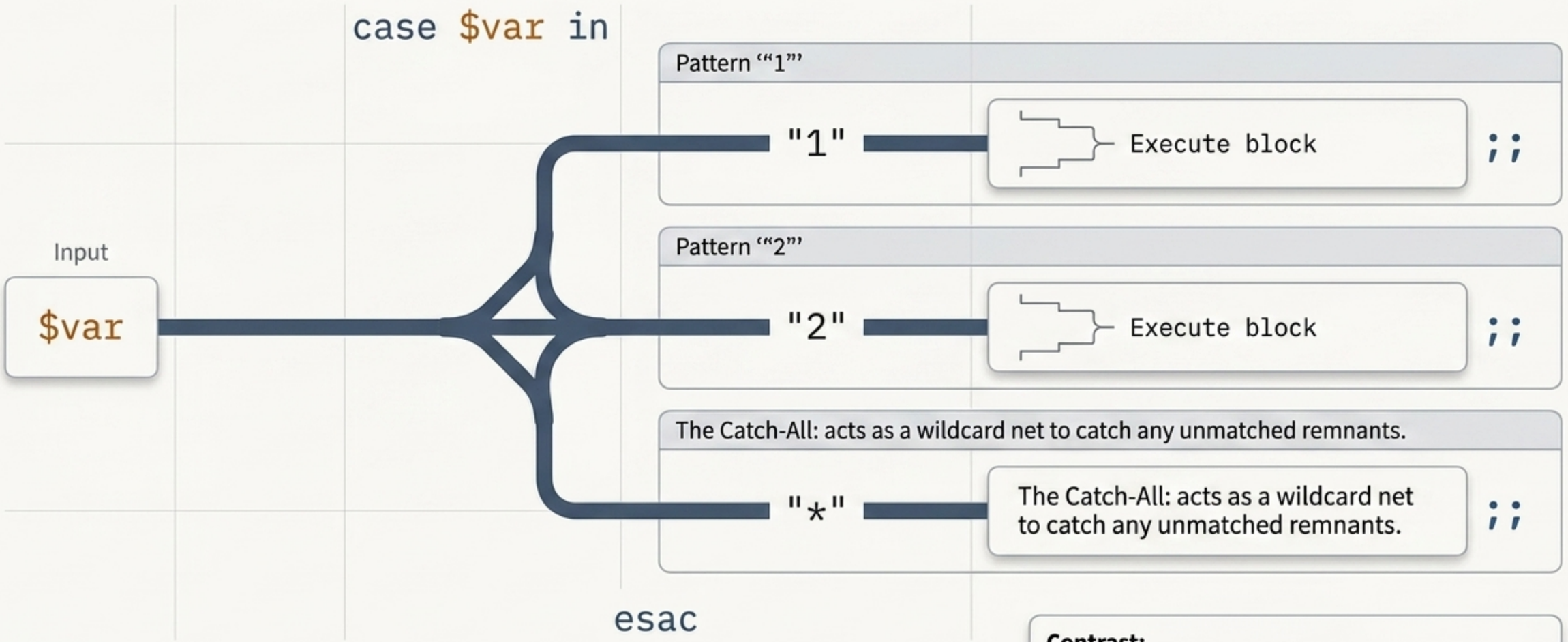
Spins constantly as long as a condition is FALSE (opposite logic to while).

`break` (exit loop immediately)

`continue` (skip current iteration)

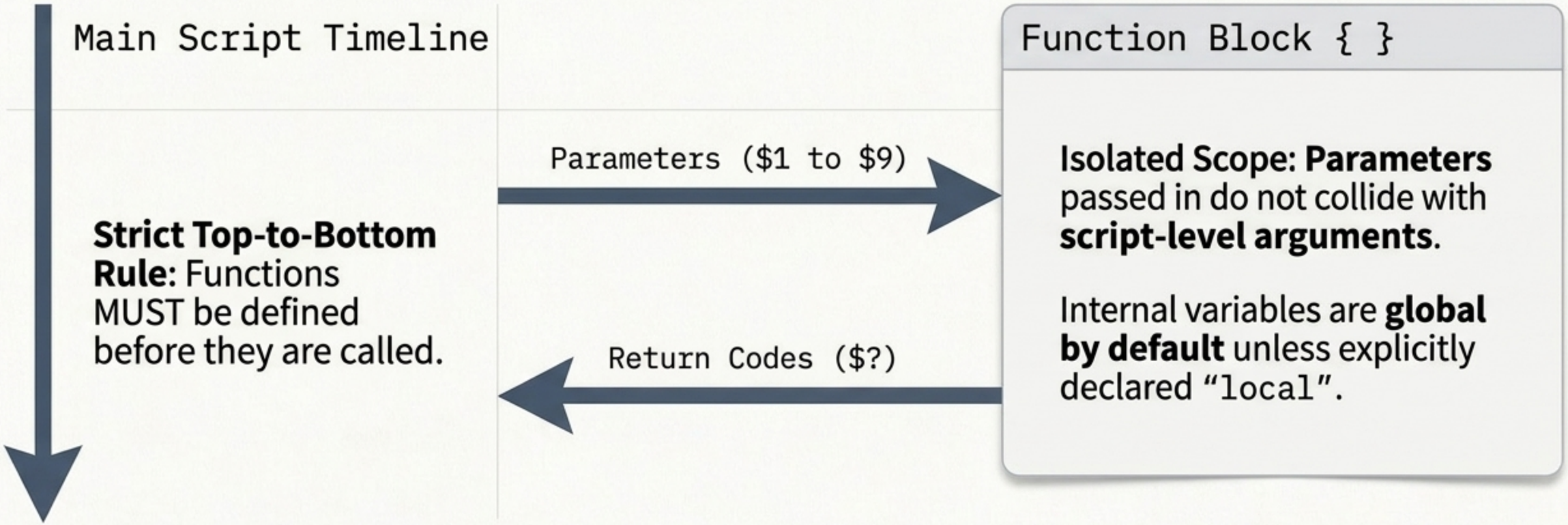
⚠ **WARNING:** While/Until loops require a counter or exit condition to prevent infinite loops.

Branching Logic: Exact Pattern Matching with Case



Contrast:
Use '**case**' for exact pattern matching.
Use '**if-else**' for boolean expressions or range comparisons.

Modular Architecture: Building Reusable Functions



Standard Return Codes		
	0 : Success	126 : Cannot execute
	1 : General error	127 : Command not found
	2 : Built-in misuse	130 : Terminated by Ctrl+C

X-Ray Vision: Tracing Execution and Debugging

`bash -x script.sh (xtrace)`

```
+ var="World"
+ echo "Hello, World"
Hello, World
+ count=5
+ '[' 5 -gt 0 ']'
+ echo "Loop iteration 1"
Loop iteration 1
```

Shows the command executed and the ACTUAL evaluated values.

`bash -v script.sh (verbose)`

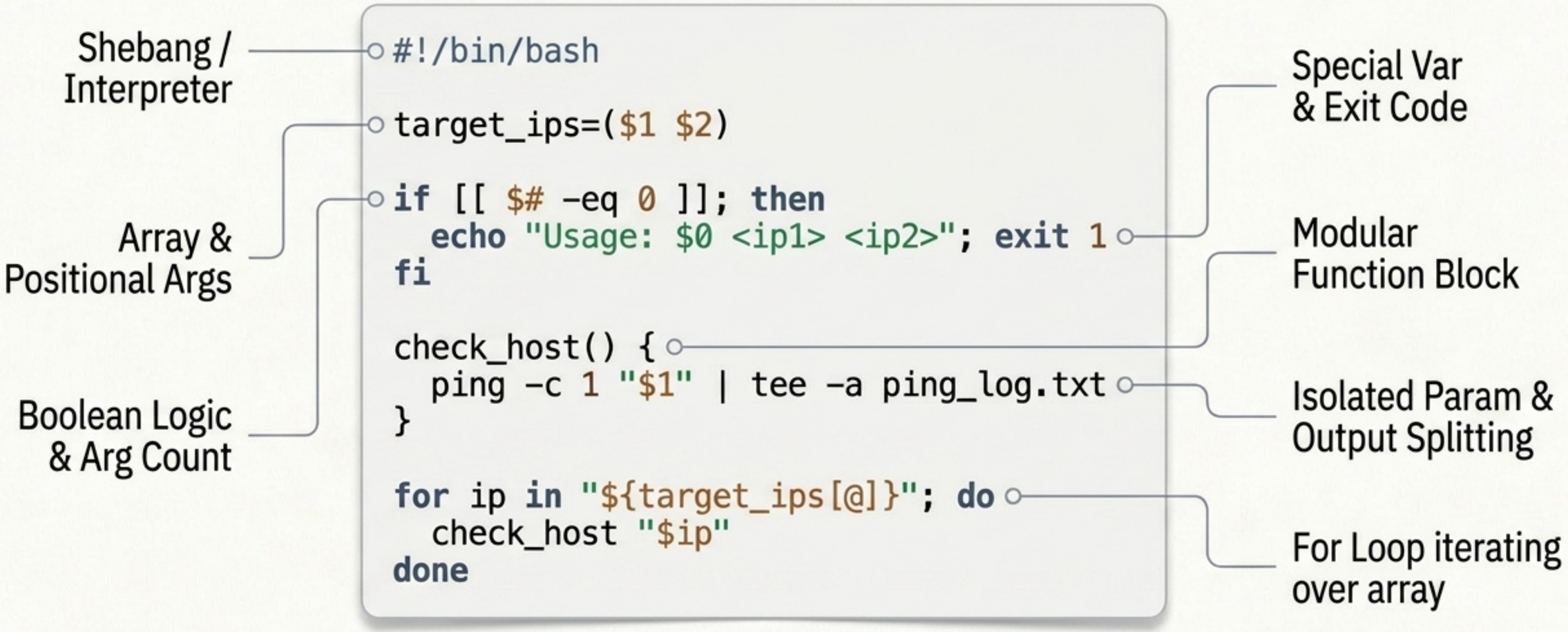
```
#!/bin/bash
var="World"
echo "Hello, $var"
count=5
while [ $count -gt 0 ]; do
  echo "Loop iteration $count"
  ((count--))
done
```

Shows the raw, unexecuted script code processed alongside the executed steps.

`-x` (Evaluated Values) + `-v` (Source Context) = `bash -x -v script.sh`

Provides complete diagnostic visibility to identify exactly where and why a script fails.

Synthesis: Anatomy of an Automated Workflow



Commands, variables, flow control, and **modular logic** perfectly combined into a **unified execution engine.**